Conversational Agents

Von der Idee zum eigenen Conversational Agent

26.08.2026

Paul-Ferdinand Steuck, Daniel Bierschwale

Einführung für das IT-Studienprojekt



Agenda

A **Grundlagen**
Was ist ein Conversational Agent, was passiert im LLM, Tokens, Prompts und Rollen

B **Zugang & Betrieb**
API oder lokal, Quantisierung, Modelle finden und bewerten

C **Steuerung & Bau**
No-Code oder Framework, Bausteine, Tool Calling, Messen (Tracing)

D **Sprache**
Zuhören und sprechen, Werkzeuge für Sprach-Agenten

E **Wissen & Gedächtnis**
RAG, Metadaten, hybride Suche, Fine-Tuning, Gedächtnis über Sitzungen

A · Grundlagen

Was ist ein Conversational Agent, was passiert im Inneren und wie redet man mit ihm?

Generative KI: der größere Rahmen

Generative KI

Systeme, die Inhalte erzeugen, die sich nicht mehr oder nur schwer von menschlich erstellten unterscheiden lassen.

Aktuell die zugänglichste und populärste Subdisziplin der Künstlichen Intelligenz.

Was sie erzeugt

Text, Bilder, Video oder Audio

Wie sie lernt

Training auf sehr großen Datenmengen, etwa tausenden Hundebildern, um Hunde erzeugen zu können

Wo sie beispielsweise eingesetzt wird

Forschung und Programmierung, Bürotätigkeiten und Prozessoptimierung

Sprachmodelle sind der Teilbereich, um den es in diesem Vortrag geht.

Large Language Models: Grundlagen

Was sind LLMs?

Die Definition

- KI-Systeme auf Basis maschineller Lernverfahren
- Entwickelt, um menschliche Sprache zu verstehen und zu erzeugen
- Trainiert auf sehr umfangreichen Textdatensätzen: Webseiten, Bücher, Code

Geschichte

Woher die Technik kommt

- Ursprünge in der Forschung zu neuronalen Netzen
- Deutlicher Fortschritt durch Deep Learning
- Transformer-Architektur 2017 als Wendepunkt
- Seit 2023 breit verfügbar und alltagstauglich

Funktionsweise

Was im Modell passiert

- Transformer erfasst den Kontext und gewichtet die wichtigen Wörter
- Daraus werden Wahrscheinlichkeiten für das Folgewort berechnet
- Die Antwort entsteht Stück für Stück, Token für Token

Wichtig: Ein LLM versteht keine Bedeutung → Es rechnet Wahrscheinlichkeiten aus. Genau daraus entsteht der Eindruck von Sprachkompetenz.

Was ist ein Token?

Modelle verarbeiten keine Wörter, sondern Tokens. Das sind Wortstücke von etwa vier Zeichen.

Ein	Agent	beant	wortet	Fra	gen

„Ein Agent beantwortet Fragen“ = etwa 6 Tokens, etwa 30 Zeichen

Daumenregeln

- Ein Token sind etwa vier Zeichen: Ein deutsches Wort besteht somit oft aus zwei Tokens
- Auch das Kontextfenster wird in Tokens gemessen, nicht in Wörtern
- Preise von Sprachmodell werden häufig pro eine Millionen Token angegeben

Abgerechnet wird pro Token

- Eingabe- und Ausgabe-Token werden getrennt gezählt, Ausgabe ist in der Regel teurer
- Jeder Aufruf schickt den ganzen Verlauf mit
  → Lange Gespräche kosten mehr

Möglichkeiten Kosten gering zu halten: Kleine Modelle für einfache Schritte, Verlauf zusammenfassen, Caching nutzen

Was ist ein Prompt und wozu dient er?

Beispiel: die Bausteine eines Prompts

Rolle „Du bist Tutor für Statistik.“

Aufgabe „Erkläre den p-Wert in drei Sätzen.“

Kontext „Zielgruppe sind Erstsemester.“

Format „Ein Absatz, keine Formeln.“

Grenze „Antworte nur aus dem beigefügten Skript.“

Antwort des Modells

„Der p-Wert sagt, wie wahrscheinlich ein so extremes Ergebnis wäre, wenn nichts dahintersteckt.“

Was einen guten Prompt ausmacht

- Rolle, Aufgabe, Format und Grenzen benennen
- Beispiele wirken stärker als Regeln
  → Zwei Musterantworten sind häufig hilfreich
- Was nicht im Prompt steht, weiß das Modell in diesem Moment nicht.
- Prompts sind essenziell
  → Versionieren und testen ist sehr wichtig
- Nachschärfen gehört dazu

Kein Training nötig, nur präzise Sprache. Wie diese Bausteine technisch getrennt werden, zeigt die nächste Folie.

Die drei Rollen im Gespräch

System-Prompt	User-Prompt	Assistant
Die Verfassung: Rolle, Regeln, Tonfall und Grenzen. Für Nutzende unsichtbar, gilt für das ganze Gespräch mit dem System.	Die eigentliche Frage oder Aufgabe: dazu gehören auch angehängte Dateien und gefundene Textstellen.	Die früheren Antworten des Modells. Werden mitgeschickt, damit der Verlauf erinnert wird.

```
messages = [
    {"role": "system",    "content": "Du bist Tutor. Antworte kurz und auf Deutsch."},
    {"role": "user",      "content": "Was ist ein Token?"},
    {"role": "assistant", "content": "Ein Token ist ein Textbaustein ..."},
    {"role": "user",      "content": "Wie viele passen ins Kontextfenster?"},
]
```

Der Prompt von der letzten Folie verteilt sich auf diese drei Rollen: system einmal, user und assistant im Wechsel.

Was ist ein Conversational Agent?

Technologische Artefakte, mit denen Nutzende über natürliche Sprache interagieren, schriftlich oder gesprochen.

Natürliche Sprache

Kein Menü und keine Suchmaske: man schreibt oder spricht wie mit einem Menschen.

Dialog im Wechsel

Frage, Antwort, Rückfrage. Der Verlauf trägt die Bedeutung, nicht die einzelne Eingabe.

Viele Namen

Chatbot, digitaler Assistent, interaktiver Agent: in der Literatur synonym verwendet.

„Julie“ von Amtrak

Beantwortet im Kundenservice rund fünf Millionen Anfragen pro Jahr.

„Amelia“ von IPsoft

Übernimmt Aufgaben im IT-Servicedesk und beantwortet Kundenanfragen.

Definition nach Diederich, Brendel, Morana und Kolbe (2022)

Conversational Agents: Erscheinungsformen

Ohne Verkörperung

Reiner Text- oder Sprachdialog, kein sichtbares Gegenüber. Beispiel: „Barista“ von Starbucks.

Statischer Avatar

Ein festes Bild oder Profilfoto als Gegenüber, ohne Bewegung.

Interaktiver Avatar

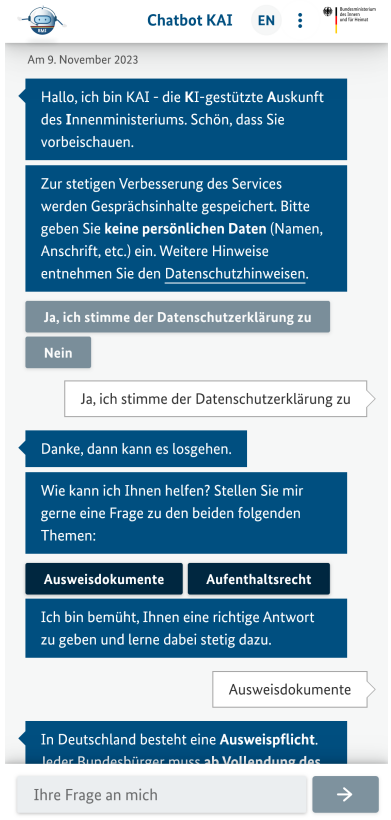
Animierte Figur mit Mimik und Stimme. Beispiel: „Amelia“ von IPsoft.

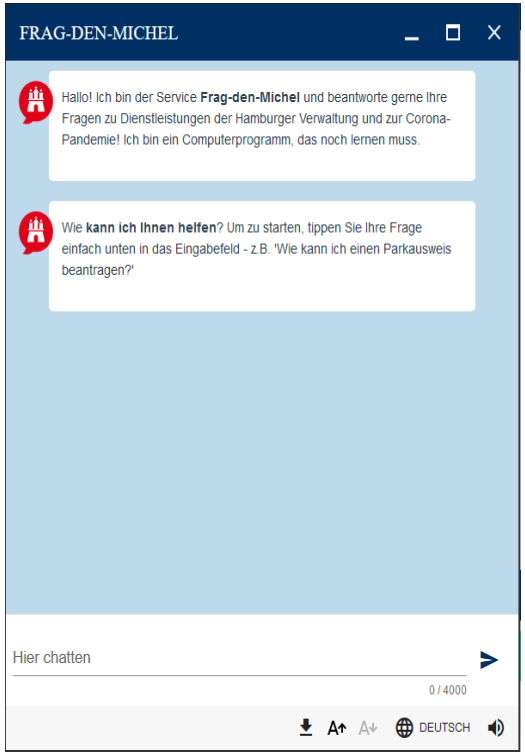
Physisch verkörpert

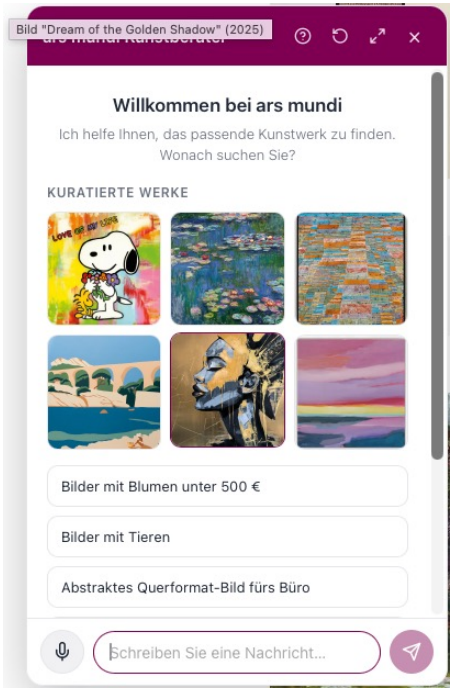
Serviceroboter im Raum, mit Sensorik. Beispiel: „Pepper“ von SoftBank.

Klassifikation nach Diederich, Brendel, Morana und Kolbe (2022)

Wo Conversational Agents heute eingesetzt werden







Large Language Models: Grenzen und Risiken

Halluzinationen	Das Modell erfindet plausibel klingende Fakten.
Trainingsbias	Trainingsdaten spiegeln die Perspektiven ihrer Quellen. Somit werden Stereotype und Einseitigkeiten mitgelernt und reproduziert.
Wissensstichtag	Die Trainingsdaten enden an einem Datum X. Neueres fehlt, solange keine weiteren Möglichkeiten für Wissensergänzung angebunden sind.
Prompt Injection	In Dokumenten oder Webseiten versteckte Anweisungen können den Agenten manipulieren.
Datenschutz	Bei API-Anbietern verlassen die Eingaben das Haus: DSGVO und Hosting-Standort prüfen sind je nach Themenfeld umso wichtiger.
Nichtdeterminismus	Was heute funktioniert, kann morgen anders antworten → Gleiche Frage, andere Ausgabe.

B · Zugang & Betrieb

Woher kommt das Modell: API, lokal?

Zwei Wege zum Modell: API oder lokal

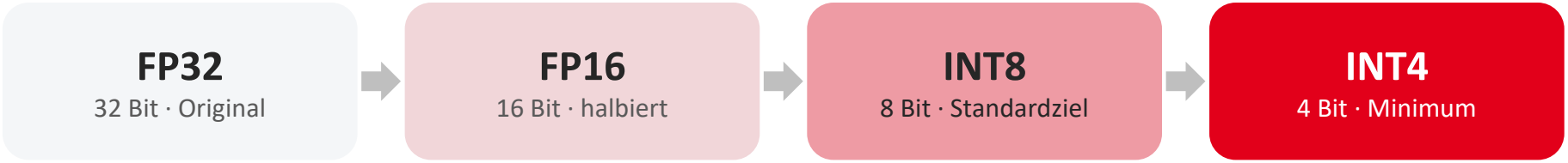
	API (Cloud)	Lokal (eigene Hardware)
Einstieg	API-Key holen und im Code hinterlegen	Modell herunterladen und installieren
Datenschutz	Alle Eingaben gehen in die Cloud	Alles bleibt im eigenen Netzwerk
Qualität	Zugriff auf die stärksten Modelle	Kleinere Modelle, je nach Hardware
Kosten	Pro Token, wächst mit der Nutzung	Einmalig Hardware, danach der eigene Strom
Gratis möglich?	Bei vielen Anbietern gibt es Testkontingente	Ja, das Modell ist dauerhaft kostenfrei

Kommerzielle Beispiele
ChatGPT (GPT-5.6), Claude Opus 5.0, Google Gemini

Open-Source-Beispiele
Qwen 3.8 - 27B, Gemma 4 - 26B, Mistral Small 4: 6-8B aktiv

Hinweis: Für den Eigenbetrieb ist eine Infrastruktur inklusive Grafikkarten notwendig.

Quantisierung: Damit es lokal funktioniert



Was dabei passiert

- Gewichte werden von FP32 oder FP16 auf INT8 oder INT4 umgerechnet
- 8 Bit ist der übliche Zielwert, 4 Bit funktioniert für viele Modelle auch

Was es bringt

- Weniger Bits pro Durchlauf → geringerer Rechenaufwand, schnellere Antworten
- Das Modell passt in kleineren Grafikspeicher und somit auch auf einem Notebook
- Die Genauigkeit bleibt ähnlich, aber ein Verlust ist nicht zu vermeiden

https://www.ibm.com/de-de/think/topics/quantization

Lokal betreiben: Werkzeuge und Hardware

Ollama

Laden und starten per Befehl, mit lokaler API

ollama.com

LM Studio

Grafische Oberfläche zum Ausprobieren

lmstudio.ai

Hugging Face

Modelle und Modellkarten: Lizenz prüfen

huggingface.co

Hardware und Formate

- Entscheidend ist der Grafikspeicher: 8 GB reichen für kleine, quantisierte Modelle
- Ohne GPU läuft es auf der CPU: langsamer, aber es läuft
- MLX-Varianten sind für Macs mit Apple Silicon gebaut
- Lokale Server sprechen die OpenAI-API: der Umstieg kostet kaum Code

Modell-Tipp

Qwen3.8-27B auf Hugging Face

huggingface.co/Qwen/Qwen3.8-27B

Modelle einschätzen und auswählen

Kennzahlen lesen

- Parametergröße (z. B. 8B, 27B, 70B) sind ein grober Hinweis auf Leistung und Hardwarebedarf
- Kontextfenster: Wie viel Text pro Anfrage hineinpasst
- Benchmarks wie MMLU oder HumanEval als Anhaltspunkt für Leistung
- Latenz und Preis pro Token: Was jede Antwort kostet

Vergleichen und auswählen

- Leaderboards für den Überblick, eigene Testfragen für die Entscheidung
- Lizenz und Hosting-Standort prüfen (DSGVO)

Selbst vergleichen

Blindvergleich zweier Modelle

arena.ai

- Zwei Modelle antworten blind auf dieselbe Frage, du wählst die bessere.
- Daraus entsteht ein Ranking, das laufend aktualisiert wird.

Kostenlos loslegen: Chat AI und LLM-Service der GWDG

Chat AI

Weboberfläche zum Ausprobieren

chat-ai.academiccloud.de

- Modelle direkt im Browser testen, ohne eine Zeile Code
- Login über die Hochschulkennung, kein Schlüssel nötig

Zum Testen empfohlen: Mistral Medium und DeepSeek V4 Flash, beide offen und mit guten Ergebnissen.

Anmeldung mit der Hochschulkennung über academiccloud.de. Welche Modelle bereitstehen, wechselt: Liste im Portal prüfen.

C · Steuerung & Bau

Frameworks, Werkzeuge und wie daraus ein Prototyp wird

Zwei Wege zum eigenen Conversational Agent

Zusammenklicken (No-Code)

- Werkzeuge wie n8n, Flowise oder Copilot Studio
- Hier verbindet man Bausteine statt Code zu schreiben
- Ideal für ein schnelles Ergebnis
- Grenzen: Sonderfälle, Tests und Versionierung sind häufig mühsam und es ist weniger flexibel

Programmieren (Code)

- z. B. in Python mit einem Agent Framework wie Pydantic AI, OpenAI Agents SDK oder LangGraph
- Volle Kontrolle über Logik, Fehlerfälle und Kosten
- Versionierbar und automatisiert testbar
- Mehr Flexibilität

Beispiele für Agent-Frameworks im Überblick

LangGraph

Ablauf als Graph mit Zuständen: Stark bei Schleifen und Verzweigungen

CrewAI

Mehrere Agenten mit Rollen arbeiten als Team an einer Aufgabe

Microsoft Agent Framework

Nachfolger von Semantic Kernel und AutoGen, stark im Azure-Umfeld

OpenAI Agents SDK

Schlank, in Minuten startklar, sehr nah an den OpenAI-Modellen

Pydantic AI

Typsichere Ausgaben: Validierte Datenstrukturen statt Freitext

Alle lösen dasselbe Problem: Aufrufe, Werkzeuge, Zustand und Fehler sauber organisieren.

Die Bausteine einer exemplarischen Conversational Agent-Anwendung

Agent

Plant die Schritte, ruft Werkzeuge auf, formuliert die Antwort

OpenAI Agents SDK · LangGraph · CrewAI · Pydantic AI

Frontend

Was Nutzende sehen

Streamlit, Chainlit, Open WebUI oder ein eigenes Web-UI

Antwort streamen, Quellen anzeigen, Verlauf behalten

Backend

Logik, Rechte, Protokoll

FastAPI (Python) oder Express (Node.js)

Hierüber werden Tools/Funktionen aufgerufen und die KI angesprochen

Vektordatenbank

Wissen zum Nachschlagen

Qdrant, Chroma oder pgvector in PostgreSQL

Speichert die Abschnitte als Vektoren samt Metadaten

Das Modell ist nur ein Baustein. Frontend, Backend und Datenbank machen daraus ein System.

Tool Calling: wie das Modell Code auslöst

1 **Nutzer fragt**
„Wie ist das Wetter in Kiel?“

2 **Modell antwortet mit Werkzeug-Wunsch**
get_weather(city=‚Kiel‘)

3 **Euer Code führt die Funktion/Tool aus**
liefert 18 °C, bewölkt

4 **Modell formuliert die Antwort**
„In Kiel sind es 18 °C.“

```
from agents import Agent, Runner, function_tool

@function_tool
def wetter(stadt: str) -> str:
    return api.get(stadt)          # echte Abfrage

agent = Agent(name="Wetter", instructions="Antworte kurz.",
              tools=[wetter])
antwort = Runner.run_sync(agent, "Wetter in Hildesheim?")
```

Wichtig: Das Modell muss Tool Calling unterstützen, nicht jedes lokale Modell kann das!

Gedächtnis: Verlauf und mehrere Sitzungen

Im laufenden Gespräch	• Der ganze Verlauf wird bei jedem Aufruf erneut mitgeschickt • Wird es zu lang: älteste Nachrichten abschneiden oder zusammenfassen
Über Sitzungen hinweg	• Verlauf in einer Datenbank ablegen und pro Nutzerkonto laden • Frameworks liefern das mit
Langzeitgedächtnis	• Wichtige Fakten als kurze Notizen speichern • Beispiel: „duzt gern, arbeitet in Kiel“

Das Modell erinnert sich nie selbst. Gedächtnis ist immer etwas, das die Anwendung mitliefert.

Messen: Tracing und Auswertung

Was ein Trace festhält

- Jeden Aufruf mit Prompt, Kontext und Antwort
- Kosten, Tokenzahl und Latenz pro Schritt
- Welche Werkzeuge der Agent benutzt hat und in welcher Reihenfolge
- Bewertungen: Daumen von Nutzenden, Regelprüfung oder ein zweites Modell als Prüfer
- Prompt-Versionen, damit Änderungen vergleichbar bleiben

Langfuse

Open Source, selbst hostbar

langfuse.com

Bezahlte Alternative mit demselben Zweck:
LangSmith, smith.langchain.com

D · Sprache

Zuhören und sprechen: was zwischen Mikrofon und Lautsprecher passiert

Sprache verstehen und sprechen: die zwei Richtungen

Sprache hinein (Erkennung)

- Whisper lokal oder Cloud-APIs übernehmen die Transkription
- Deutsch funktioniert gut: Dialekt und Fachbegriffe deutlich schlechter
- Erkennen, wann der Sprecher fertig ist (Voice Activity Detection)
- Erkennungsfehler wandern ungefiltert in den Prompt

Sprache heraus (Ausgabe)

- Sprachausgabe über OpenAI, ElevenLabs oder Piper (lokal)
- Antwort streamen statt auf den ganzen Text zu warten
- Kurze, gesprochene Sätze statt Bullet-Listen formulieren
- Alternativ Speech-to-Speech-Modelle ohne Zwischentext

Latenz entscheidet: unter einer Sekunde Antwortzeit fühlt sich ein Sprachdialog natürlich an.

Werkzeuge für Sprache

Zuhören

Spracherkennung

- Whisper läuft lokal, kostenlos und mit guter deutscher Qualität
- Voxtral von Mistral ist offen und selbst hostbar

Sprechen

Sprachausgabe

- Dienste von OpenAI oder ElevenLabs klingen natürlich
- Piper läuft lokal auf einfacher Hardware

Verbinden

Gesprächslogik

- Pipecat verbindet Mikrofon, Modell und Ausgabe
- Kümmert sich um Unterbrechen und Sprechpausen

Kürzer geht es auch
Speech-to-Speech-Modelle nehmen Audio und antworten direkt mit Audio, ohne Text dazwischen. Weniger Verzögerung, dafür schwerer zu kontrollieren und zu protokollieren.

github.com/pipecat-ai/pipecat

E · Wissen & Gedächtnis

Wie der Agent an Wissen kommt, das nicht im Modell steckt

Drei Wege, dem Modell Wissen zu geben

Alles, was das Modell bei einer Anfrage wissen soll, muss in das Kontextfenster: dorthin führen drei Wege.

System-Prompt

Wissen direkt in die Anweisung schreiben.

Sofort gemacht, aber begrenzt und bei jedem Aufruf mitbezahlt.

Gut für: kleine, stabile Regelwerke

RAG

Zur Frage passende Textstellen suchen und mitgeben.

Aktuell, belegbar, der Standardweg für Dokumente.

Gut für: viele oder wechselnde Dokumente

Finetuning

Das Modell mit eigenen Beispielen nachtrainieren.

Prägt Stil und Format.

Gut für: Tonfall und festes Ausgabeformat sowie starres Wissen

Kurze Entscheidungshilfe

Feste, kurze Regeln	Aktuelles, belegbares Wissen	Stil, Format, Fachsprache
System-Prompt	**RAG**	**Fine-Tuning**

RAG: Idee und Nutzen

Was RAG ist

Retrieval-Augmented Generation

- Verbindet ein LLM mit dem Abruf relevanter Informationen aus externen Quellen
- Daten aus PDF, Word, Excel, PowerPoint oder Datenbanken
- Das Modell selbst bleibt unverändert

Welches Problem es löst

Gegen falsche Antworten

- Weniger Halluzinationen und erfundene Quellen
- Antworten lassen sich auf Dokumente zurückführen
- Neues Wissen ohne erneutes Training

Wie es bereitsteht

Verfügbarkeit

- Kommerziell und als Open Source verfügbar
- Gängiger Fall: intern mit den eigenen Daten chatten
- Der Aufwand liegt in der Datenaufbereitung, nicht im Modell

RAG ändert nicht das Modell, sondern das, was dem Modell zur Frage vorgelegt wird.

RAG-Bausteine: Werkzeuge und Code

Vektordatenbanken

- Qdrant: Open Source, selbst hostbar
- Pinecone: gehostet, wenig Aufwand
- Chroma: schnell zum Ausprobieren
- pgvector: direkt in PostgreSQL

Embedding-Modelle

- OpenAI text-embedding-3
- bge-m3: offen, mehrsprachig
- multilingual-e5: gut für Deutsch
- Wichtig: immer dasselbe Modell nutzen

Daten aufbereiten

- Docling: PDF zu strukturiertem Text
- Unstructured: viele Dateiformate
- Tabellen und Bilder gesondert behandeln
- Abschnitte mit Überlappung schneiden

Docling

Wandelt PDF, Word und PowerPoint verlässlich in strukturierten Text um: der Schritt vor dem Zerlegen.

github.com/docling-project/docling

Metadaten und Payload
qdrant.tech/documentation

Vektordatenbanken
qdrant.tech/articles

Die Bausteine sind Standard: zerlegen, einbetten, suchen, mit Quellen antworten.

RAG: eigene Dokumente nutzbar machen

1 · Einmal vorbereiten (Indexierung)

Laden
PDF, Web, Notizen: z. B. mit Docling

→

Zerteilen
in Abschnitte mit Überlappung

→

Einbetten
Text wird zu Zahlenvektoren

→

Speichern
Vektor-DB mit Metadaten

↓ *Der Index entsteht einmal: danach wird er bei jeder Frage wiederverwendet.*

2 · Bei jeder Frage (Retrieval)

Frage einbetten
gleiches Embedding-Modell

→

Suchen
Top-k ähnliche Treffer

→

Prompt bauen
Abschnitte + Frage ans Modell

→

Antworten
mit Quelle und Seitenzahl

Metadaten beim Speichern mitgeben: Dateiname, Seite, Datum, Kapitel. Nur so kann der Agent am Ende seine Quelle nennen, und nur so lassen sich Treffer sinnvoll filtern.

Hybride Suche und Metadaten: Ein Beispiel

Anfrage: „Wie viele Urlaubstage habe ich im ersten Jahr?“

Stichwortsuche

Findet genau dieses Wort. Stark bei Namen, Nummern und Fachbegriffen.

„… stehen 30 Urlaubstage pro Kalenderjahr zu …“

Treffer, weil „Urlaubstage“ wörtlich vorkommt

Vektorsuche

Findet die Bedeutung, auch mit anderen Wörtern.

„… Erholungsurlaub beträgt 30 Arbeitstage …“

Treffer, obwohl das Wort „Urlaubstage“ fehlt

Beide Trefferlisten werden zusammengeführt, die besten fünf Abschnitte gehen an das Modell.

Metadaten filtern vorher

```
quelle = "Betriebsvereinbarung"   und   gueltig_ab <= heute   und   sprache = "de"
```

Filter über Metadaten ermöglichen regelbasierte Filter einzubauen.

RAG weiterdenken: mehrere Runden, Belege, Agentic Search

Multi-Step-RAG

- Die Frage wird zerlegt: erst suchen, dann mit dem Gefundenen gezielt nachsuchen
- Mehrere Runden statt eines Treffers, Lücken werden nachgeladen
- Langsamer und teurer, dafür belastbar bei mehrteiligen Fragen

RAG mit Quellverweisen

- Jeder Abschnitt trägt Metadaten: Datei, Seite, Kapitel, Datum
- Das Modell zitiert die Fundstelle, die Antwort wird nachprüfbar
- Ohne Verweis ist eine RAG-Antwort nicht kontrollierbar

qdrant.tech/documentation/manage-data/payload

Mistral Agentic Search

mistral.ai/news/agentic-search

- Das Modell durchsucht das Web selbst und entscheidet, wonach es sucht
- Quellen kommen mit der Antwort zurück
- Kein eigener Index, kein Chunking nötig

Der Werkzeugkasten aus dieser Vorstellung

Modelle und Betrieb

Ollama | LM Studio | Hugging Face | Chat AI der GWDG | Qwen | Mistral | DeepSeek

Quantisierung | INT8 / INT4 | MLX für Macs

Bauen

OpenAI Agents SDK | LangGraph | CrewAI | Pydantic AI | n8n | Flowise | Dify | Tool Calling

MCP | FastAPI | Streamlit | Chainlit | Open WebUI

Wissen und Gedächtnis

RAG | Docling | Embeddings | Qdrant | Chroma | pgvector | Chunking | Metadaten

Hybride Suche | Multi-Step-RAG | Quellverweise | Agentic Search | Fine-Tuning

Sprache und Qualität

Whisper | Voxtral | Piper | Pipecat | Text-to-Speech | Speech-to-Speech | Tracing | Langfuse

LangSmith | Tokens | Kontextfenster